

28 | Nom：用Rust写一个Parser解析器

唐刚 · Rust 语言从入门到实战



你好，我是 Mike。今天我们来一起学习如何用 Rust 写一个 Parser 解析器。

说到解析器，非计算机科班出身的人会一脸懵，这是什么？而计算机科班出身的人会为之色变，曾经熬夜啃“龙书”的痛苦经历浮现眼前。解析器往往跟“编译原理”这个概念一起出现，谈解析器色变完全可以理解。

实际上，解析器也没那么难，并不是所有需要“解析”的地方都与编程语言相关。因此我们可以先把“编译原理”的负担给卸掉。在开发过程中，其实经常会碰到需要解析的东西，比如自定义配置文件，从网络上下载下来的一些数据文件、服务器日志文件等。这些其实不需要很深的背景知识。更加复杂一点的，比如网络协议的处理等等，这些也远没有达到一门编程语言的难度。

另一方面，虽然我们这门课属于入门级，但是对于未来的职业规划来说，如果你说你能写解析器，那面试官可能会很感兴趣。所以这节课我会从简单的示例入手，让你放下恐惧，迈上“解析”之路。

解析器是什么？

解析器其实很简单，就是把一个字符串或字节串解析成某种类型。对应的，在 Rust 语言里就是把一个字段串解析成一个 Rust 类型。一个 Parser 其实就是一个 Rust 函数。

这个转换过程有很多种方法。

1. 最原始的是完全手撸，一个字符一个字符吞入解析。
2. 对一些简单情况，直接使用 String 类型中的 find、split、replace 等函数就可以。
3. 用正则表达式能够解析大部分种类的文本。
4. 还可以用一些工具或库帮助解析，比如 Lex、Yacc、LalrPop、Nom、Pest 等。
5. Rust 语言的宏也能用来设计 DSL，能实现对 DSL 文本的解析。

这节课我们只关注第 4 点。在所有辅助解析的工具或库里，我们只关心 Rust 生态辅助解析的库。

Rust 生态中主流的解析工具

目前 Rust 生态中已经有几个解析库用得比较广泛，我们分别来了解下。

🔗 **LalrPop** 类似于 Yacc，用定义匹配规则和对应的行为方式来写解析器。

🔗 **Pest** 使用解析表达式语法（Parsing Expression Grammar, PEG）来定义解析规则，PEG 已经形成了一个成熟的标准，各种语言都有相关的实现。

Nom 是一个解析器组合子（Parser-Combinator）库，用函数组合的方式来写规则。一个 Parser 就是一个函数，接收一个输入，返回一个结果。而组合子 combinator 也是一个函数，用来接收多个 Parser 函数作为输入，把这些小的 Parser 组合在一起，形成一个大的 Parser。这个过程可以无限叠加。

Nom 库介绍

这节课我们选用 Nom 库来讲解如何快速写出一个解析器，目前（2023 年 12 月）Nom 库的版本为 v7.1。选择 Nom 的原因是，它可以用来解析几乎一切东西，比如文本协议、二进制文件、流数据、视频编码数据、音频编码数据，甚至是一门完整功能的编程语言。

Nom 的显著特性在安全解析、便捷的解析过程中的错误处理和尽可能的零拷贝上。因此用 Nom 解析库写的代码是非常高效的，甚至比你用 C 语言手撸一个解析器更高效，这里有一些 [评测](#) 你可以参考。Nom 能够做到这种程度主要是因为站在了 Rust 的肩膀上。

解析器组合子是一种解析方法，这种方法不同于 PEG 通过写单独的语法描述文件的方式进行解析。Nom 的 slogan 是 “nom, eating data byte by byte”，也就是一个字节一个字节地吞，顺序解析。

使用 Nom 你可以写特定目的的小函数，比如获取 5 个字节、识别单词 HTTP 等，然后用有意义的模式把它们组装起来，比如识别 `'HTTP'`，然后是一个空格、一个版本号，也就是 `'HTTP 1.1'` 这种形式。这样写出的代码就非常小，容易起步。并且这种形式明显适用于流模式，比如网络传输的数据，一次可能拿不完，使用 Nom 能够边取数据边解析。

解析器组合子思路有 5 个优势。

解析器很小，很容易写。

解析器的组件非常容易重用。

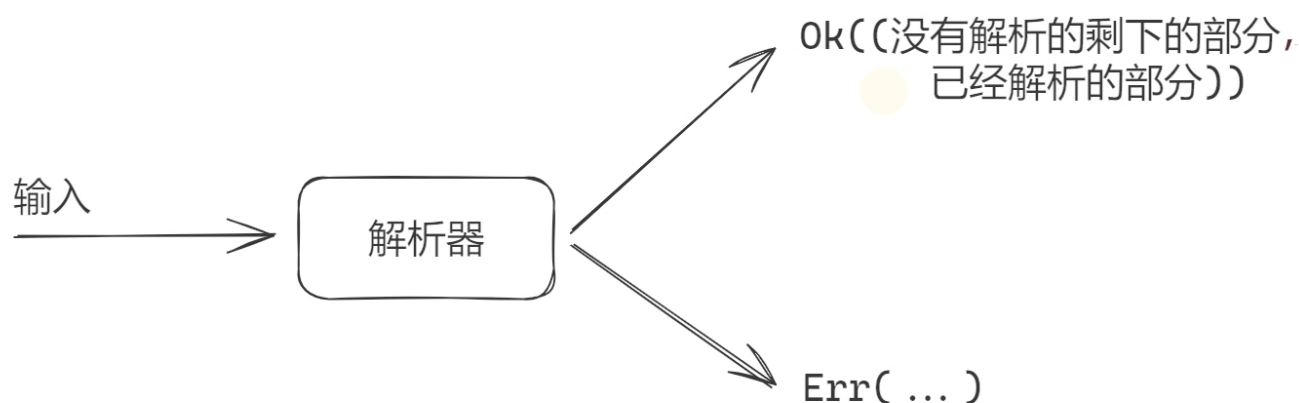
解析器的组件非常容易用单元测试进行独立测试。

解析器组合的代码看起来接近于你要解析的数据结构，非常直白。

你可以针对你当前的特定数据，构建部分解析器，而不用担心其他数据部分。

Nom 的工作方式

Nom 的解析器基本工作方式很简单，就是读取输入数据流，比如字符串，返回 `(rest, output)` 这样一个 tuple，rest 就是没有解析到的字符串的剩余部分，output 就是解析出来的目标类型。很多时候，这个返回结果就是 `(&str, &str)`。解析过程中，可以处理解析错误。



基本解析器和组合子

在 Nom 中，一个 Parser 其实就是一个函数。Nom 提供了一些最底层的 Parser。相当于构建房屋的砖块，我们掌握了这些砖块后，就可以把这些砖块组合使用，像乐高积木，一层层越搭越高。

这里我们列举一些常用的解析器，案例基本上都是对字符串的解析。

Tag

tag 非常常用，用来指代一个确定性的字符串，比如 "hello"。

🔗 tag：识别一个确定性的字符串。

🔗 tag_no_case：识别一个确定性的字符串，忽略大小写。

基本类别解析器

下面是 Nom 提供的用来识别字符的基本解析器，可以看到，都是我们熟知的解析器。

🔗 alpha0：识别 a-z, A-Z 中的字符 0 个或多个。

🔗 alpha1：识别 a-z, A-Z 中的字符 1 个或多个（至少 1 个）。

🔗 alphanumeric0：识别 0-9, a-z, A-Z 中的字符 0 个或多个。

- 🔗 **alphanumeric1**: 识别 0-9, a-z, A-Z 中的字符 1 个或多个 (至少 1 个) 。
- 🔗 **digit0**: 识别 0-9 中的字符 0 个或多个。
- 🔗 **digit1**: 识别 0-9 中的字符 1 个或多个 (至少 1 个) 。
- 🔗 **hex_digit0**: 识别 0-9, A-F, a-f 中的字符 0 个或多个。
- 🔗 **hex_digit1**: 识别 0-9, A-F, a-f 中的字符 1 个或多个 (至少 1 个) 。
- 🔗 **space0**: 识别 空格和 tab 符 \t 0 个或多个。
- 🔗 **space1**: 识别 空格和 tab 符 \t 0 个或多个 (至少 1 个) 。
- 🔗 **multispace0**: 识别 空格、tab 符 \t、回车符 \r、换行符 \n, 0 个或多个。
- 🔗 **multispace1**: 识别 空格、tab 符 \t、回车符 \r、换行符 \n, 1 个或多个 (至少 1 个) 。
- 🔗 **tab**: 识别确定的制表符 \t。
- 🔗 **newline**: 识别确定的换行符 \n。
- 🔗 **line_ending**: 识别 '\n' 和 '\r\n' 。
- 🔗 **not_line_ending**: 识别 '\n' 和 '\r\n' 之外的其他字符 (串) 。
- 🔗 **one_of**: 识别给定的字符集合中的一个。
- 🔗 **none_of**: 识别给定的字符集合之外的字符。

完整的列表请看这里：

🔗 <https://docs.rs/nom/latest/nom/character/complete/index.html>

基本组合子

- 🔗 **alt**: Try a list of parsers and return the result of the first successful one 或组合子，满足其中的一个解析器就可成功返回。
- 🔗 **tuple**: 和组合子，并且按顺序执行解析器，并返回它们的值为一个 tuple。

🔗 **delimited**: 解析左分界符目标信息右分界符这种格式，比如 `"{ ... }"`，返回目标信息。

🔗 **pair**: tuple 的两元素版本，返回一个二个元素的 tuple。

🔗 **separated_pair**: 解析目标信息分隔符目标信息这种格式，比如 `"1,2"` 这种，返回一个二个元素的 tuple。

🔗 **take_while_m_n**: 解析最少 m 个，最多 n 个字符，这些字符要符合给定的条件。

更多 Nom 中的解析器和组合子的信息请查阅 🔗 **Nom 的 API**。

Nom 实战

我们从最简单的解析器开始。

0 号解析器

0 号解析器就相当于整数的 0，这是一个什么也干不了的解析器。

 复制代码

```
1 use std::error::Error;
2 use nom::IResult;
3
4 pub fn do_nothing_parser(input: &str) -> IResult<&str, &str> {
5     Ok((input, ""))
6 }
7
8 fn main() -> Result<(), Box<dyn Error>> {
9     let (remaining_input, output) = do_nothing_parser("abcdefg")?;
10    assert_eq!(remaining_input, "abcdefg");
11    assert_eq!(output, "");
12    Ok(())
13 }
```

上面的 `do_nothing_parser()` 函数就是一个 Nom 的解析器，对，就是一个普通的 Rust 函数，它接收一个 `&str` 参数，返回一个 `IResult<&str, &str>`，`IResult<I, O>` 是 Nom 定义的解析器的标准返回类型。你可以看一下它的 🔗 **定义**。

```
1 pub type IResult<I, O, E = Error<I>> = Result<(I, O), Err<E>>;
```

可以看到，正确返回情况下，它的返回内容是 `(I, O)`，一个元组，元组第一个元素是剩下的未解析的输入流部分，第二个元素是解析出的内容。这正好对应 `do_nothing_parser()` 的返回内容 `(input, "")`。这里是原样返回，不做任何处理。

注意，`E = Error<I>` 这种写法是类型参数的默认类型，请回顾课程 [第 10 讲](#) 找到相关知识。

看起来这个解析器没有啥作用，但不可否认，它让我们直观感受了 Nom 中的 parser 是个什么东西，我们已经有了基本模板。

1 号解析器

这次我们必须要做点什么事情了，那就把 `"abcdefg"` 的前三个字符识别出来。我们需要用到 tag 解析器。代码如下：

```
1 pub use nom::bytes::complete::tag;
2 pub use nom::IResult;
3 use std::error::Error;
4
5 fn parse_input(input: &str) -> IResult<&str, &str> {
6     tag("abc")(input)
7 }
8
9 fn main() -> Result<(), Box<dyn Error>> {
10     let (leftover_input, output) = parse_input("abcdefg")?;
11     assert_eq!(leftover_input, "defg");
12     assert_eq!(output, "abc");
13
14     assert!(parse_input("defdefg").is_err());
15     Ok(())
16 }
```

在这个例子里，`tag("abc")` 的返回值是一个 `parser`，然后这个 `parser` 再接收 `input` 的输入，并返回 `IResult<&str, &str>`。前面的我们看到，`tag` 识别固定的字符串 / 字节串。

`tag` 实际返回一个闭包，你可以看一下它的定义。

[复制代码](#)

```
1 pub fn tag<T, Input, Error: ParseError<Input>>(<br>2     tag: T<br>3 ) -> impl Fn(Input) -> IResult<Input, Input, Error><br>4 where<br>5     Input: InputTake + Compare<T>,<br>6     T: InputLength + Clone,
```

也就是返回下面这行内容。

[复制代码](#)

```
1 impl Fn(Input) -> IResult<Input, Input, Error>
```

这里这个 `Fn` 就是用于描述闭包的 `trait`，你可以回顾一下课程 [第 11 讲](#) 中关于它的内容。

这个示例里 `parse_input("abcdefg")`？这个解析器会返回 `("defg", "abc")`，也就是把 `"abc"` 解析出来了，并返回了剩下的 `"defg"`。而如果在待解析输入中找不到目标 `pattern`，那么就会返回 `Err`。

解析一个坐标

下面我们再加大难度，解析一个坐标，也就是从 `"(x, y)"` 这种形式中解析出 `x` 和 `y` 两个数字来。

代码如下：

[复制代码](#)

```
1 use std::error::Error;<br>2 use nom::IResult;
```

```
3 use nom::bytes::complete::tag;
4 use nom::sequence::{separated_pair, delimited};
5
6 #[derive(Debug, PartialEq)]
7 pub struct Coordinate {
8     pub x: i32,
9     pub y: i32,
10 }
11
12 use nom::character::complete::i32;
13
14 // 解析 "x, y" 这种格式
15 fn parse_integer_pair(input: &str) -> IResult<&str, (i32, i32)> {
16     separated_pair(
17         i32,
18         tag(", "),
19         i32
20     )(input)
21 }
22
23 // 解析 "( ... )" 这种格式
24 fn parse_coordinate(input: &str) -> IResult<&str, Coordinate> {
25     let (remaining, (x, y)) = delimited(
26         tag("("),
27         parse_integer_pair,
28         tag(")")
29     )(input)?;
30
31     Ok((remaining, Coordinate {x, y}))
32 }
33 }
34
35 fn main() -> Result<(), Box<dyn Error>> {
36     let (_, parsed) = parse_coordinate("(3, 5)")?;
37     assert_eq!(parsed, Coordinate {x: 3, y: 5});
38
39     let (_, parsed) = parse_coordinate("(2, -4)")?;
40     assert_eq!(parsed, Coordinate {x: 2, y: -4});
41
42     // 用nom, 可以方便规范地处理解析失败的情况
43     let parsing_error = parse_coordinate("(3,)");
44     assert!(parsing_error.is_err());
45
46     let parsing_error = parse_coordinate("(,3)");
47     assert!(parsing_error.is_err());
48
49     let parsing_error = parse_coordinate("Ferris");
50     assert!(parsing_error.is_err());
51 }
```



```
52     Ok(())
53 }
```

我们从 `parse_coordinate() parser` 看起。首先遇到的是 `delimited` 这个 combinator，它的作用我们查一下上面的表格，是解析左分界符目标信息右分界符这种格式，返回目标信息，也就是解析 `(xxx)`，`<xxx>`，`{xxx}` 这种前后配对边界符的 pattern，正好可以用来识别我们这个 `(x, y)`，我们把 `"(x, y)"` 第一步分解成 `"("，"x, y"，")"` 三部分，用 `delimited` 来处理。同样的，它也返回一个解析器闭包。

然后，对于中间的这部分 `"x, y"`，我们用 `parse_integer_pair()` 这个 parser 来处理。继续看这个函数，它里面用到了 `separated_pair` 这个 combinator。查一下上面的表，你会发现它是用来处理左目标信息分隔符右目标信息这种 pattern 的，刚好能处理我们的 `"x, y"`。中间那个分隔符就用一个 `tag(", ")` 表示，两侧是 `i32` 这个 parser。注意，这里这个 [i32](#) 是代码中引入的。

[复制代码](#)

```
1 use nom::character::complete::i32;
```

不是 Rust std 中的那个 `i32`，它实际是 `Nom` 中提供的一个 parser，用来把字符串解析成 std 中的 `i32` 数字。`separated_pair` 也返回一个解析器闭包。可以看到，返回的闭包调用形式和 `delimited` 是一样的。其实整个 `Nom` 解析器的签名都是固定的，可以以这种方式无限搭积木。

`parse_integer_pair` 就返回了 `(x, y)` 两个 `i32` 数字组成的元组类型，最后再包成 `Coordinate` 结构体类型返回。整个任务就结束了。

可以看到，这实际就是标准的**递归下降**解析方法。先识别大 pattern，分割，一层层解析小 pattern，直到解析到最小单元为止，再组装成需要的输出类型，从函数中一层层返回。整个过程就是普通的 Rust 函数栈调用过程。

解析 16 进制色彩编码

下面我们继续看一个示例：解析网页上的色彩格式 #2F14DF。

对于这样比较简单的问题，手动用 String 的方法分割当然可以，用正则表达式也可以。这里我们来研究用 Nom 怎样做。

[复制代码](#)

```
1 use nom::{
2     bytes::complete::{tag, take_while_m_n},
3     combinator::map_res,
4     sequence::Tuple,
5     IResult,
6 };
7
8 #[derive(Debug, PartialEq)]
9 pub struct Color {
10     pub red: u8,
11     pub green: u8,
12     pub blue: u8,
13 }
14
15 fn from_hex(input: &str) -> Result<u8, std::num::ParseIntError> {
16     u8::from_str_radix(input, 16)
17 }
18
19 fn is_hex_digit(c: char) -> bool {
20     c.is_digit(16)
21 }
22
23 fn hex_primary(input: &str) -> IResult<&str, u8> {
24     map_res(take_while_m_n(2, 2, is_hex_digit), from_hex)(input)
25 }
26
27 fn hex_color(input: &str) -> IResult<&str, Color> {
28     let (input, _) = tag("#")(input)?;
29     let (input, (red, green, blue)) = (hex_primary, hex_primary, hex_primary).par
30     Ok((input, Color { red, green, blue }))
31 }
32
33 #[test]
34 fn parse_color() {
35     assert_eq!(
36         hex_color("#2F14DF"),
37         Ok((
38             "",
39             Color {
40                 red: 47,
```

```

41         green: 20,
42         blue: 223,
43     }
44 })
45 );
46 }
```

执行 `cargo test`, 输出：

 复制代码

```

1  running 1 test
2  test parse_color ... ok
3
4  test result: ok. 1 passed; 0 failed; 0 ignored; 0 measured; 0 filtered out; finis
```

我们来详细解释这个文件。

代码从 `hex_color` 入手, 输入就是 `"#2F14DF"` 这个字符串。

 复制代码

```
1 let (input, _) = tag("#")(input)?;
```

这句执行完, 返回的 `input` 变成 `"2F14DF"`。

接下来就要分析三个 16 进制数字, 两个字符一组。

 复制代码

```
1 (hex_primary, hex_primary, hex_primary).parse(input)?;
```

我们在元组上直接调用了 `.parse()` 函数。这是什么神奇的用法？别慌，你在 `std` 标准库文档里面肯定找不到，实现在 [这里](#)。它将常用的元组变成了 `parser`。但是这样的实现需要手动调用一下 `.parse()` 函数来执行解析。

这里我们意图就是把颜色解析成独立的三个元素，每种元素是一个 16 进制数，这个 16 进制数进一步用 `hex_primary` 来解析。我们再来看 `hex_primary` 的实现。

[复制代码](#)

```
1 map_res(  
2     take_while_m_n(2, 2, is_hex_digit),  
3     from_hex  
4 )(input)
```

其中，代码第二行表示在 `input` 中一次取 2 个字符（前面两个参数 2, 2，表示返回不多于 2 个，不少于 2 个，因此就是等于 2 个），取出每个字符的时候，都要判断是否是 16 进制数字。是的话才取，不是的话就会返回 `Err`。

`map_res` 的意思是，对 `take_while_m_n parser` 返回的结果应用一个后面提供的函数，这里就是 `from_hex`，它的目的是把两个 16 进制的字符组成的字符串转换成 10 进制数字类型，这里就是 `u8` 类型。因此 `hex_primary` 函数返回的结果是 `IResult<&str, u8>`。
`u8::from_str_radix(input, 16)` 是 Rust std 库中的 `u8` 类型的自带方法，16 表示 16 进制。

[复制代码](#)

```
1 let (input, (red, green, blue)) = (hex_primary, hex_primary, hex_primary).parse
```

因此这一行，正常返回后，`input` 就为 `""` 了，`(red, green, blue)` 这三个是 `u8` 类型的三元素 `tuple`，实际这里相当于定义了 `red`、`green`、`blue` 三个变量。

然后下面一行，就组装成 `Color` 对象返回了，目标完成。

[复制代码](#)

```
1 Ok((input, Color { red, green, blue }))
```

更多示例

前面我们说过，Nom 非常强大，可应用领域非常广泛，这里有一些链接，你有兴趣的话，可以继续深入研究。

解析 HTTP2 协议：🔗 <https://github.com/sozu-proxy/sozu/blob/main/lib/src/protocol/h2/parser.rs>

解析 flv 文件：🔗 <https://github.com/rust-av/flavors/blob/master/src/parser.rs>，你还可以对照 C 实现体会 Nom 的厉害之处：

🔗 <https://github.com/FFmpeg/FFmpeg/blob/master/libavformat/flvdec.c>。

解析 Python 代码：🔗 <https://github.com/progval/rust-python-parser>

自己写一个语言：🔗 <https://github.com/Rydgel/monkey-rust>

小结

这节课我们学习了如何用 Nom 解决解析器任务。在计算机领域，需要解析的场景随处可见，以前的 lexer、yacc 等套路其实已经过时了，Rust 的 Nom 之类的工具才是业界最新的成果，你掌握了 Nom 等工具，就能让这类工作轻松自如。

我们需要理解 Nom 这类解析器库背后的**解析器 - 组合子**思想，它是一种通用的解决复杂问题的构建方法，也就是递归下降分解问题，从上到下分割任务，直到问题可解决为止。然后先解决基本的小问题，再把这些成果像砖块那样组合起来，于是便能够解决复杂的系统问题。

可以看到，Nom 的学习门槛其实并不高，其中很关键的一点是学完一部分就能应用一部分，不像其他有些框架，必须整体学完后才能应用。一旦你通过一定的时间掌握了 Nom 的基本武器零件后，就会收获到一项强大的新技能，能够让你在以后的工作中快速升级，解决你以前不敢去解决的问题。

这节课你应该也能感受到 Rust 打下的坚实基础（安全编程、高性能等），Rust 生态已经构建出强大框架和工具，这些框架和工具能够让我们达到前所未有的生产力水平，已经完全不输于甚至超过其他编程语言了。

这节课所有可运行代码在这里:

🔗 <https://github.com/miketang84/jikeshijian/tree/master/28-nom>

思考题

请尝试用 Nom 解析一个简单版本的 CSV 格式文件。欢迎你把你解析的内容分享出来，我们一起看一看，如果你觉得这节课对你有帮助的话，也欢迎你分享给其他朋友，我们下节课再见!

© 版权归极客邦科技所有，未经许可不得传播售卖。页面已增加防盗追踪，如有侵权极客邦将依法追究其法律责任。

全部留言 (3)

最新 精选



Promise

2023-12-27 来自北京

老师，请问解析器如何优化性能，比如解析器每天需要处理 PB 级别的文本。每个文件 100M 并且需要匹配上百种规则。

作者回复: 100M小问题的，一次性读入内存就行。不同的规则的话，可以用 alt

<https://docs.rs/nom/latest/nom/branch/fn.alt.html>，另外，nom 还支持流解析，边产生边解析。



tianyu0901

2023-12-27 来自上海

老师，推荐一个SQL解析器

作者回复: <https://crates.io/crates/sqlparser>

共 2 条评论 >



superggn

2024-01-05 来自北京

思考题

...

```

use nom::{
    bytes::complete::is_not,
    character::complete::{char, line_ending},
    combinator::opt,
    multi::separated_list0,
    sequence::terminated,
    IResult,
};

use std::{fs, io::Error};

fn read_file(file_path: &str) -> Result<String, Error> {
    fs::read_to_string(file_path)
}

// parse_csv => parser
fn parse_csv(input: &str) -> IResult<&str, Vec<Vec<&str>>> {
    println!("input csv file:");
    println!("{}", input);
    // terminated => combinator
    // line_ending => parser
    // opt => combinator
    separated_list0(terminated(line_ending, opt(line_ending)), parse_line)(input)
}

// parse_line => parser
fn parse_line(input: &str) -> IResult<&str, Vec<&str>> {
    // separated_list0 => a combinator
    // accepts 2 parser
    separated_list0(char(','), is_not("\r\n"))(input)
}

fn main() -> Result<(), Box<dyn std::error::Error>> {
    let file_content = read_file("/path/to/my/file.csv"?);
    match parse_csv(&file_content) {
        Ok((_, rows)) => {
            for row in rows {

```



```
        println!("{:?}", row);
    }
}
Err(e) => println!("Failed to parse CSV: {:?}", e),
}
Ok(())
}
` ``
```

